



Complete Windows Desktop Application Specification
End-to-End Build Prompt, Debug Protocol & Installer Recipe

Target	Windows 10 (21H2+) / Windows 11 — x64
Stack	Electron 28 + React 18 + TypeScript + Vite + Tailwind
AI Backend	Ollama (local LLM) — no cloud tokens required
Media Engine	yt-dlp + ffmpeg (bundled inside installer)
Auth	Google OAuth 2.0 (YouTube Data API v3)
Output	NSIS installer (.exe) + portable .exe
Audience	Non-technical end user — zero coding required
Token budget	Single Claude Code session prompt — zero follow-ups

Instructions for Claude Code: Read this entire document before writing any code. Implement every feature, run the multi-phase debug protocol in Section 13, fix ALL issues found, then produce the signed Windows NSIS installer. Do not ask the user for API keys or tokens — all credentials are entered by the end user at runtime through the Settings UI.

■ **Copyright notice:** Re-uploading other creators' content may violate YouTube's Terms of Service and applicable copyright law. The end user accepts full responsibility. This tool is provided for legitimate use with content the user owns or has explicit permission to republish.

Table of Contents

1. Project Overview & Goals	3
2. Tech Stack (exact versions)	3
3. Project Directory Structure	4
4. Feature #1 — Google OAuth Login	6
5. Feature #2 — Single Video Upload by URL	7
6. Feature #3 — Batch Upload an Entire Channel	8
7. Feature #4 — Auto-Sync (Monitor & Auto-Upload)	9
8. Feature #5 — Local-LLM AI Clipping (Shorts)	10
9. Database Schema (SQLite)	14
10. IPC / Process Architecture	15
11. UI/UX Specification	17
12. Error Handling, Retry & Quota	19
13. Mandatory Multi-Phase Debug Protocol	20
14. Build, Bundle & Installer	22
15. Acceptance Checklist	25
16. Final Instructions to Claude Code	26

1. PROJECT OVERVIEW & GOALS

Build a **Windows desktop application** that lets a user:

- **Log in with their own Google account** (OAuth 2.0) and grant YouTube upload/read scopes.
- **Paste a single YouTube video URL** from another channel -> the app downloads that video (best quality) and re-uploads it to the user's own YouTube channel -- fully automated, no manual steps after confirmation.
- **Paste a channel URL/handle** -> the app enumerates all public videos on that channel, lets the user pick which ones, and batch-downloads + re-uploads them.
- **Auto-sync mode**: the user adds one or more channels to a "watch list"; a background monitor checks on a configurable schedule (1h / 6h / 12h / 24h); every new video uploaded to those channels is automatically downloaded and either (a) queued for approval or (b) immediately uploaded to the user's channel, depending on user setting.
- **AI Clipping**: the user provides a long video (URL or local file); a **local LLM running via Ollama** identifies the most viral/highlight/jump-scare/controversial moments; the app auto-edits each highlight into a vertical 9:16 YouTube Short (with auto-captions, zoom-punch, background music, and auto-thumbnail), shows previews in the UI, and on user approval uploads each short to the user's channel.

Non-goals / constraints:

- The app **MUST NOT** require the user to write any code, run any terminal commands, or manually edit config files.
- The app **MUST** work fully offline for the AI clipping step (local Ollama; no cloud LLM calls).
- The installer **MUST** be a single `.exe` (NSIS) that bundles all required binaries (ffmpeg, ffprobe, yt-dlp).
- The app **MUST** run on Windows 10 (21H2+) and Windows 11, 64-bit only.
- **Copyright warning (display in app on first run)**: Re-uploading others' content may violate YouTube's TOS and copyright law; the user accepts full responsibility.

2. TECH STACK (exact)

Component	Choice	Notes
Runtime	Electron 28+ (latest stable 28.x)	Main + renderer processes
Renderer framework	React 18 + TypeScript 5	Strict mode on
Build tooling	Vite 5 + <code>electron-vite</code>	Fast HMR in dev
UI library	TailwindCSS 3.4 + shadcn/ui (Radix)	Dark-first theme, red YouTube accent (#FF0000)
Icons	<code>lucide-react</code>	
State management	Zustand 4	Single stores folder
Routing	<code>react-router-dom</code> v6	Hash router for Electron
Notifications (toast)	<code>sonner</code>	
Forms	<code>react-hook-form</code> + <code>zod</code>	
Local DB	<code>better-sqlite3</code> v9	Synchronous, fast

Component	Choice	Notes
Credential storage	keytar	Windows Credential Manager
YouTube API	googleapis v126+ (youtube v3)	Resumable uploads
Google OAuth	google-auth-library	OAuth2 with localhost redirect
YouTube download	yt-dlp-exec + bundled yt-dlp.exe	Best-quality muxed MP4
Video processing	fluent-ffmpeg + bundled ffmpeg.exe/ffprobe.exe	Cut, scale, subtitles, audio mix
Local LLM	http://localhost:11434 (Ollama)	JSON-mode for structured output
Transcription	(a) YouTube auto-captions via yt-dlp; (b) fallback: bundled whisper.cpp (or leave as optional download if size prohibits bundling)	
Scheduling	node-cron v3	Channel monitor jobs
System tray	Electron native Tray	Minimize-to-tray, status badge
HTTP client	axios	Ollama, misc
Image processing (thumbnails)	sharp v0.33	
Dates (Persian/Jalali)	jalaali-js	Optional; default Gregorian
Packaging	electron-builder v24	NSIS installer, portable exe
Fonts	Inter (Latin) + Vazirmatn (Persian)	Bundled locally, no Google CDN

3. PROJECT DIRECTORY STRUCTURE

```
youtube-auto-uploader/  
+- package.json  
+- tsconfig.json  
+- tsconfig.node.json  
+- vite.config.ts  
+- electron.vite.config.ts  
+- tailwind.config.js  
+- postcss.config.js  
+- components.json  
+- .env.example  
+- resources/  
| +- icon.ico  
| +- icon.png  
| +- fonts/  
| | +- Inter-Regular.ttf  
| | +- Inter-Bold.ttf  
| | +- Vazirmatn-Regular.ttf  
| +- music/  
| | +- background_lofi.mp3  
| +- binaries/  
| | +- ffmpeg.exe  
| | +- ffprobe.exe  
| | +- yt-dlp.exe  
+- electron/  
| +- main.ts  
| +- preload.ts
```

```
# shadcn/ui config  
# only for build-time; NO runtime secrets here  
  
# Windows app icon (256x256 multi-size)  
# Tray icon  
  
# Royalty-free lofi loop (embed 1 short sample ~30s)  
# Downloaded by prebuild script (see Section 14)  
  
# Main process entry  
# contextBridge: expose safe API to renderer
```

```

| +- tray.ts                # System tray integration
| +- updater.ts             # (optional stub) auto-update check
| +- bin.ts                 # Resolves paths to bundled ffmpeg/yt-dlp
| +- ipc/
| | +- auth.ipc.ts          # Google OAuth flows
| | +- youtube.ipc.ts       # YouTube API wrappers (list, upload, quota)
| | +- downloader.ipc.ts    # yt-dlp wrapper with progress
| | +- uploader.ipc.ts      # Upload queue manager
| | +- monitor.ipc.ts       # Channel monitor / auto-sync scheduler
| | +- clipper.ipc.ts        # AI clipping orchestrator
| | +- ffmpeg.ipc.ts        # ffmpeg helpers (cut, scale, sub, mix)
| | +- ollama.ipc.ts        # Ollama HTTP client + health check
| | +- whisper.ipc.ts       # Transcription fallback
| | +- thumbnail.ipc.ts     # Thumbnail generation with sharp
| | `-- settings.ipc.ts     # Get/set settings
| +- services/
| | +- AuthService.ts
| | +- YouTubeService.ts
| | +- DownloadService.ts
| | +- UploadQueue.ts
| | +- ChannelMonitor.ts
| | +- ClipperEngine.ts
| | +- FFmpegService.ts
| | +- OllamaService.ts
| | +- WhisperService.ts
| | +- ThumbnailService.ts
| | +- SettingsService.ts
| | `-- NotificationService.ts
| +- db/
| | +- index.ts             # Opens better-sqlite3, runs migrations
| | +- migrations/
| | | `-- 001_init.sql
| | `-- repositories/
| | | +- channels.repo.ts
| | | +- videos.repo.ts
| | | +- clips.repo.ts
| | | +- history.repo.ts
| | | `-- settings.repo.ts
| | `-- utils/
| | | +- logger.ts          # File + console logger
| | | +- retry.ts           # Exponential-backoff helper
| | | +- paths.ts           # App data / temp dirs
| | | `-- oauth-server.ts   # Local HTTP server for OAuth callback
| `-- src/                  # Renderer (React)
| | +- main.tsx
| | +- App.tsx
| | +- index.css            # Tailwind + @font-face declarations
| | +- i18n/
| | | +- en.json
| | | `-- fa.json          # Persian translations
| | +- lib/
| | | +- api.ts             # Wrapper over window.api (preload)
| | | +- utils.ts           # cn(), formatters
| | | `-- store.ts          # Zustand stores (auth, queue, settings)
| | +- components/
| | | +- ui/                # shadcn/ui generated components
| | | +- Sidebar.tsx
| | | +- TopBar.tsx
| | | +- VideoCard.tsx
| | | +- VideoPlayer.tsx    # HTML5 custom player
| | | +- ClipCard.tsx
| | | +- ProgressBar.tsx
| | | +- UploadForm.tsx
| | | +- ChannelListItem.tsx
| | | +- OAuthSetupGuide.tsx # Walkthrough for Client ID creation
| | | `-- Onboarding.tsx
| | `-- pages/

```

```
+ - Login.tsx
+ - Dashboard.tsx
+ - UploadSingle.tsx
+ - UploadChannel.tsx
+ - AutoSync.tsx
+ - Clipper.tsx
+ - PendingApproval.tsx
+ - History.tsx
+ - Settings.tsx
```

4. FEATURE #1 -- GOOGLE OAUTH LOGIN

4.1 Credential model

- Do **NOT** embed any Client ID / Client Secret in code. On first launch the Settings page shows a guided wizard: 1. Opens a link to `https://console.cloud.google.com/apis/credentials` 2. Shows screenshots/step-by-step text for: create project -> enable YouTube Data API v3 -> create "OAuth 2.0 Client ID" (Desktop application type) -> add `http://localhost:5173/oauth2callback` (dev) and `http://127.0.0.1:{dynamicPort}/callback` (prod) to Authorized redirect URIs. 3. Two masked inputs for Client ID and Client Secret with "Test & Save" button.
- Credentials stored via `keytar` under service name `YouTubeUploader/GoogleClientId` etc.
- Encrypted fallback to `app.getPath('userData')/secure.dat` (AES-256-GCM via Node crypto with machine-id key) if `keytar` unavailable.

4.2 OAuth flow

- User clicks **"Sign in with Google"** (large, centered, Google-branded button).
- Main process starts a temporary local HTTP server on a random free port (`127.0.0.1:0`) with a single `/callback` route.
- Builds auth URL with scopes:
 - `https://www.googleapis.com/auth/youtube.upload`
 - `https://www.googleapis.com/auth/youtube.readonly`
 - `https://www.googleapis.com/auth/youtube.force-ssl`
 - `email`
 - `profile`
- Opens `shell.openExternal(authUrl)` in user's default browser.
- On callback, exchanges code for tokens; stores `access_token`, `refresh_token`, `expiry_date` in `keytar`.
- Fetches user profile (`oauth2.userInfo.get`) and channel info (`channels.list(mine=true, part=snippet, statistics)`) and persists to DB settings.
- IPC sends `auth:state-changed` event to renderer -- app navigates to Dashboard.

4.3 Session

- On app start, if tokens exist and are expired, refresh automatically.
- If refresh fails (revoked), force logout and show Login page.
- Sidebar shows: profile picture (64px circular), channel name, email, sign-out button.

5. FEATURE #2 -- SINGLE VIDEO UPLOAD BY URL

5.1 Input

- Prominent paste-area (large textarea with paste-icon) + "Paste from clipboard" button.
- URL validation (regex): accepts `youtube.com/watch?v=`, `youtu.be/`, `youtube.com/shorts/`, `m.youtube.com/`.
- "Fetch info" button auto-triggers on valid paste (debounced 500ms).

5.2 Metadata preview

After URL validation, app runs `yt-dlp -j <url>` to fetch JSON metadata and displays:

- Thumbnail (maxres default, fall back to hq -> mq)
- Title (editable input)
- Original channel name + link
- Duration (mm:ss)
- View count (formatted)
- Upload date
- Available qualities dropdown (default: `bestvideo[height<=1080]+bestaudio/best`; user can override to 720p/1440p/2160p)

5.3 Upload form

Editable before upload:

- Title (pre-filled, supports template tokens `{original_title}`)
- Description (textarea, template editor)
- Tags (chip input, comma-separated)
- Category (dropdown: 1-Film, 2-Autos, 10-Music, 15-Pets, 17-Sports, 20-Gaming, 22-People, 23-Comedy, 24-Entertainment, 25-News, 26-Howto, 27-Education, 28-Science)
- Privacy status: Public / Unlisted / Private (default from Settings)
- "Made for kids" toggle (default: off)
- Thumbnail override (file picker; default = source thumbnail)

5.4 Execution flow

- User clicks **"Download & Upload"**.
- Create job in DB (status `downloading`).
- Invoke `yt-dlp` with:

```
`` yt-dlp -f "<quality>" --merge-output-format mp4 \ --write-thumbnail
--convert-thumbnails jpg \ -o "<tempDir>/%(id)s.%(ext)s" <url> ` Parse progress
lines to emit download:progress` IPC events (percent, speed, ETA).
```

- On download complete, move to status `uploading`.
- Call YouTube Data API `videos.insert` with `part=snippet`, `status` using resumable upload (chunk size 10 MB, report progress per chunk).
- After successful upload, if custom thumbnail: `thumbnails.set(videoId, ...)`.
- Status -> `success`; show Windows notification + in-app toast; delete temp files (unless Settings says keep); write `upload_history` row; open preview modal with embedded YouTube player.
- Show "Copy link", "Open on YouTube", "Upload another" buttons.

6. FEATURE #3 -- BATCH UPLOAD AN ENTIRE CHANNEL

6.1 Input & resolution

- Accepts: `@handle`, `youtube.com/@handle`, `youtube.com/c/Name`, `youtube.com/channel/UC...`, `youtube.com/playlist?list=...`.
- `yt-dlp` resolves to a playlist ID; app fetches metadata for ALL videos via pagination (`--flat-playlist -J`) with progress.
- Display total count + estimated total size.

6.2 Selection UI

- Virtualized list (use `@tanstack/react-virtual`) with each row: checkbox, thumbnail, title, duration, views, date.
- Bulk actions: Select All, Select None, Select last N, Select date range.
- Search/filter box (by title).
- Per-item title edit (inline) and privacy override (menu).

6.3 Batch execution

- Queue-based concurrency (default 2 parallel downloads and 1 parallel upload, configurable in Settings).
- Global progress at the top: X of Y complete; Pause / Resume / Cancel buttons.
- Per-row status pill: ■ Queued · ■ ■ Downloading · ■ ■ Uploading · ■ Done · ■ Failed (with retry button) · ■ ■ Skipped.
- Failures retried automatically (see Section 12) up to 3 times, then marked failed with reason.
- On completion: summary modal (success count, fail count, total time, CSV export of failures).

7. FEATURE #4 -- AUTO-SYNC (MONITOR & AUTO-UPLOAD)

7.1 Channel management

- "Add Channel" dialog with URL input (resolves like Section 6.1).
- Per-channel settings:
 - Check interval (1h / 6h / 12h / 24h / custom cron)
 - Auto-upload: On (instant) / Off (queue for approval)
 - Quality override
 - Title/description templates (override global)
 - Tags append
 - Privacy override
 - "Ignore shorts" / "Only shorts" toggle
 - Active/Paused switch

7.2 Monitor engine (`ChannelMonitor.ts`)

- Registers a `node-cron` job per active channel at its interval, OR uses a single tick-every-15-min job that dispatches due channels (recommended for simplicity).
- On each tick: 1. Fetch latest uploads playlist ID via `channels.list(contentDetails) -> relatedPlaylists.uploads`. 2. Call `playlistItems.list(playlistId=uploads, maxResults=20, order=date)` -- compare video IDs against `synced_videos` table. 3. For every new video (most recent first), insert a row with `status='pending'` and queue the download+upload job (or move to PendingApproval queue if auto-upload is off). 4. Update `last_checked_at`.
- ETag/If-None-Match support to reduce quota usage.
- First-run behavior: do NOT backfill the entire channel history; only videos uploaded AFTER the moment the channel was added are auto-synced (with a one-time tooltip explaining this + optional "Backfill last N" button).

7.3 Lifecycle

- Starts automatically on app launch (if Settings `autoStartMonitor = true`).
- Runs even when window is closed (minimized to tray).
- Tray icon changes: gray = idle; blue = checking; green = uploading; red = error.
- Windows toast notification on each new detected video; click opens the app on that job's detail.
- Exponential backoff for per-channel failures (back-off 1h -> 4h -> 12h -> next scheduled run).

8. FEATURE #5 -- LOCAL-LLM AI CLIPPING (HIGHLIGHTS / SHORTS)

8.1 Input page

- Two tabs: **YouTube URL** and **Local File**.
- Options panel:
 - LLM model dropdown (populated by GET `http://localhost:11434/api/tags`; mark recommended models with a star: `qwen2.5:14b-instruct`, `llama3.1:8b-instruct`, `gemma2:9b-instruct`).
 - Number of clips: slider 3-20 (default 5).
 - Max clip length: 15 / 30 / 60 / 90 sec (default 60).
 - Content category: Sports / Gaming / Educational / Vlog / Comedy / News / Challenge / Auto-detect.
 - Output aspect: 9:16 (Shorts/Reels/TikTok) · 1:1 · 16:9 (original).
 - Visual effects: ✓ auto-captions (burned-in) · ✓ smart zoom (Ken Burns / face-centered punch-in) · ✓ background music (volume slider, default 12%) · ✓ blurred background fill for portrait.
- Auto-upload on approval checkbox (default off -- user reviews each clip).

8.2 Prerequisites check

Before start, run these checks (show friendly status cards with fix buttons):

- **Ollama reachable:** GET `http://localhost:11434/api/tags` with 2s timeout. If fail -> "Download Ollama" button opening `https://ollama.com/download/windows` + "Retry" button.
- **Model available:** if the selected model is not in tags, offer to `ollama pull <model>` via streaming progress in the UI (`stream /api/pull`).
- **ffmpeg present:** verify bundled binaries run (`ffmpeg -version`).
- **Disk space:** warn if free space < 5 GB in temp dir.

8.3 Pipeline (ClipperEngine.ts)

Step 1 -- Download / load video (same DownloadService as Section 5).

Step 2 -- Audio analysis (ffmpeg):

- Extract mono 16kHz WAV: `ffmpeg -i in.mp4 -vn -ac 1 -ar 16000 audio.wav`
- Compute loudness/energy in 1-second windows with `ebur128` filter + `astats` to obtain per-second RMS peaks.
- Detect silence: `silencedetect=n=-30dB:d=0.8` to find pauses (used as safe cut points).

Step 3 -- Scene detection (ffmpeg):

- `ffmpeg -i in.mp4 -filter:v "select='gt(scene,0.35)',showinfo" -f null - 2> scenes.log`
- Parse log to produce list of `(timestamp, scene_score)` tuples.

Step 4 -- Transcription:

- **Primary:** try `yt-dlp --write-auto-sub --sub-langs en.*,fa.* --skip-download -o subs <url>`; if found, parse `.vtt` to array of `{start, end, text}`.
- **Fallback:** if Whisper binary bundled and model file present, run `whisper.cpp -m ggml-base.bin -f audio.wav -osrt -of out`; otherwise call Ollama if a whisper-family model is installed; otherwise show warning and use only scene+audio signals (still works, but less accurate).

Step 5 -- Segment construction:

- Build candidate segments by splitting at scene changes and silence boundaries; cap each candidate at `max_clip_length` seconds.
- Score each candidate by:
 - `audio_peak_score` (normalized RMS max) -> 0-1
 - `scene_change_score` (density) -> 0-1
 - `speech_rate_score` (words per second from transcript, higher = more dynamic) -> 0-1
 - `keyword_score` from transcript (regex for: "wait", "oh my god", "wow", "look", "guys", "literally", "insane", "crazy", "never", "best", "worst", plus Persian equivalents if language detected).
- Keep top N segments (`N = number_of_clips x 3`) as candidates for LLM review.

Step 6 -- LLM analysis:

- Build prompt: `` You are an expert short-form video editor. Given the transcript below and a list of candidate segments from a long-form video, select EXACTLY {N} clips that will be most engaging / viral as YouTube Shorts. Prioritize: emotional spikes, surprise, controversy, punchlines, game-winning moments, revelations, calls to action, funny moments. Content category: {category}. Maximum clip duration: {max_len} seconds.

For each chosen clip return JSON with:

- start: number (seconds, must match a candidate start exactly)
- end: number (seconds)
- score: 1-10 (virality estimate)
- title: string (`<=70` chars, hook-style, no clickbait lies)
- hook: string (first 3 words the viewer sees; must grab attention)
- hashtags: array of 3-5 relevant hashtags (no spaces, with #)
- reason: one-sentence rationale

Return ONLY a valid JSON object of shape `{ "clips": [...] }` -- no prose, no markdown fences.

TRANSCRIPT: `{transcript_with_timestamps}`

CANDIDATE SEGMENTS (start, end, audio_score, scene_score, keywords): {candidates_json} ``

- POST to `http://localhost:11434/api/generate` with `{model, prompt, stream:false, format:"json", options:{temperature:0.7, num_predict:2048}}`.
- Validate JSON (`JSON.parse + zod schema {clips: Array<{start:number,end:number,score:number,title:string,hook:string,hashtags:string[],reason:string}>}`); if invalid, retry up to 2 times with a stricter repair prompt.
- Merge LLM picks with audio/visual signals; clamp durations to `max_len`; snap cuts to nearest silence boundary within $\pm 2s$.

Step 7 -- Render each clip with ffmpeg:

- Cut lossless-cut first: `ffmpeg -ss {start-0.3} -to {end+0.3} -i in.mp4 -c:v libx264 -preset medium -crf 20 -c:a aac -b:a 192k -movflags +faststart raw_clip.mp4`
- Apply 9:16 portrait pipeline (if aspect=9:16):
- Stack filter: `split[orig][bg];[bg]scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,boxblur=20:1[bgf];[orig]scale=900:-1,format=yuv420p[fg];[bgf][fg]overlay=(W-w)/2:(H-h)/2`
- If "smart zoom" enabled: add subtle zoompan or `scale+crop` that slowly zooms 100% -> 110% over clip length.
- Burn captions:
- Build `.srt` file from transcript words/phrases that fall inside `[start,end]` (chunk at ≤ 5 words per cue, phrase on word boundaries).
- Style: `subtitles=clip.srt:force_style='FontName=Inter Bold,FontSize=22,PrimaryColour=&H00FFFFFF,OutlineColour=&H00000000,BorderStyle=1,Outline=3,Shadow=1,MarginV=110,Alignment=2'`
- For Persian: use `Vazirmatn` and set `FontName=Vazirmatn`.
- Mix background music:
- Loop bundled `background_lofi.mp3` to clip length; duck music during loud speech (use `sidechaincompress` filter with original audio as input, `threshold=-20dB`, `ratio=4`).
- Final mix: `[voice][music]amix=inputs=2:duration=first:weights=1 0.15[out]`
- Output: `final_clip_{i}.mp4` (faststart, yuv420p).

Step 8 -- Auto-thumbnail:

- Pick the frame with highest `scene_score` within the clip, or the frame at `start + (end-start)*0.3`.
- Extract with `ffmpeg`; composite via `sharp`: 1080x1920 gradient overlay at bottom, bold title text (70 chars max, 2 lines, white with black stroke), small channel badge top-left.
- Save `thumb_{i}.jpg`.

Step 9 -- Persist clips in DB (`clips` table, status `pending`) and push to renderer via IPC as each clip finishes (user sees results streaming in, not waiting for all).

8.4 Preview & approval UI

- Grid of `ClipCards`; each shows:
- Inline HTML5 player (loop muted on hover; click to play with sound).
- Title (editable), hashtags (editable chips), virality score badge.
- Generated thumbnail preview.
- Duration, file size.

- Buttons: ■ Approve · —■ Re-edit (reopen in trim modal) · ■ Save As... (local export) · ■ Regenerate title · ■■ Discard.
- Floating action bar: "Approve all", "Upload approved", "Discard all", "Export all to folder".
- Approve -> status `approved`; if auto-upload is on, immediately enqueue to UploadQueue with `type=clipper`.

8.5 Upload of shorts

- Use same upload flow as Section 5 but:
- Append `#Shorts` to tags (YouTube requirement for Shorts detection).
- Privacy defaults per Settings.
- Set `selfDeclaredMadeForKids = false`.

9. DATABASE SCHEMA (SQLite)

File: `electron/db/migrations/001_init.sql`

```
PRAGMA foreign_keys = ON;
```

```
CREATE TABLE IF NOT EXISTS settings (
  key   TEXT PRIMARY KEY,
  value TEXT NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS user_account (
  id                INTEGER PRIMARY KEY CHECK (id=1),
  google_id         TEXT,
  email             TEXT,
  name              TEXT,
  picture_url       TEXT,
  channel_id        TEXT,
  channel_name      TEXT,
  channel_thumb     TEXT,
  subscriber_count  INTEGER DEFAULT 0,
  video_count       INTEGER DEFAULT 0,
  access_token      TEXT,
  refresh_token     TEXT,
  expiry_date       INTEGER
);
```

```
CREATE TABLE IF NOT EXISTS channels (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  youtube_channel_id TEXT UNIQUE NOT NULL,
  channel_name      TEXT,
  channel_handle    TEXT,
  thumbnail_url     TEXT,
  uploads_playlist_id TEXT,
  interval_hours    INTEGER DEFAULT 6,
  auto_upload       INTEGER DEFAULT 0, -- 0=queue for approval, 1=instant
  quality           TEXT DEFAULT 'best',
  title_template    TEXT DEFAULT '{original_title}',
  desc_template     TEXT DEFAULT '{original_description}',
  extra_tags        TEXT DEFAULT '',
  privacy           TEXT DEFAULT 'unlisted',
  ignore_shorts     INTEGER DEFAULT 0,
  only_shorts       INTEGER DEFAULT 0,
  is_active         INTEGER DEFAULT 1,
  backfill_done     INTEGER DEFAULT 0,
  last_checked_at   TEXT,
  last_error        TEXT,
);
```

```
    created_at          TEXT DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS synced_videos (
  id                    INTEGER PRIMARY KEY AUTOINCREMENT,
  channel_id            INTEGER NOT NULL REFERENCES channels(id) ON DELETE CASCADE,
  source_video_id       TEXT NOT NULL,
  source_title          TEXT,
  source_url            TEXT,
  source_channel_name   TEXT,
  downloaded_path       TEXT,
  thumbnail_path        TEXT,
  uploaded_video_id     TEXT,
  uploaded_url          TEXT,
  upload_type           TEXT DEFAULT 'single', -- single | batch | auto_sync | clipper
  privacy               TEXT DEFAULT 'unlisted',
  status                TEXT DEFAULT 'pending', -- pending | downloading | uploading | success | failed | approved
  error_message         TEXT,
  attempts              INTEGER DEFAULT 0,
  file_size_mb          REAL,
  clip_id               INTEGER,
  created_at            TEXT DEFAULT (datetime('now')),
  updated_at            TEXT DEFAULT (datetime('now')),
  UNIQUE(channel_id, source_video_id, upload_type)
);

CREATE TABLE IF NOT EXISTS clips (
  id                    INTEGER PRIMARY KEY AUTOINCREMENT,
  source_video_id       TEXT,
  source_title          TEXT,
  source_path           TEXT,
  start_time            REAL NOT NULL,
  end_time              REAL NOT NULL,
  score                 INTEGER,
  suggested_title       TEXT,
  hook                  TEXT,
  hashtags              TEXT, -- JSON array
  reason                TEXT,
  local_path            TEXT,
  thumbnail_path        TEXT,
  status                TEXT DEFAULT 'pending', -- pending | approved | rejected | uploaded
  uploaded_video_id     TEXT,
  uploaded_url          TEXT,
  created_at            TEXT DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS upload_history (
  id                    INTEGER PRIMARY KEY AUTOINCREMENT,
  source_url            TEXT,
  source_title          TEXT,
  uploaded_id           TEXT,
  uploaded_url          TEXT,
  upload_type           TEXT,
  status                TEXT,
  error                 TEXT,
  size_mb               REAL,
  duration_sec          REAL,
  created_at            TEXT DEFAULT (datetime('now'))
);

CREATE TABLE IF NOT EXISTS job_log (
  id                    INTEGER PRIMARY KEY AUTOINCREMENT,
  job_type              TEXT,
  ref_id                INTEGER,
  level                 TEXT,
  message               TEXT,
  created_at            TEXT DEFAULT (datetime('now'))
);
```

```
);
```

```
CREATE INDEX IF NOT EXISTS idx_synced_videos_status ON synced_videos(status);
CREATE INDEX IF NOT EXISTS idx_clips_status          ON clips(status);
CREATE INDEX IF NOT EXISTS idx_channels_active       ON channels(is_active);
```

Migrations: simple version-numbered SQL files; on init, run all in name order; track applied migrations in `settings` key `schema_version`.

10. IPC / PROCESS ARCHITECTURE

10.1 Process split

- **Main process:** ALL heavy work (network, filesystem, ffmpeg, yt-dlp, SQLite, Ollama, scheduler, tray).
- **Renderer:** ONLY UI rendering. No Node integration. No remote module.
- **Preload** uses `contextBridge.exposeInMainWorld('api', { ... })` with a STRICT allow-list of channels.

10.2 Preload API shape (TypeScript interface in `electron/preload.ts`)

```
export interface IpcAPI {
  // auth
  auth: {
    getState: () => Promise<AuthState>;
    startLogin: () => Promise<void>;
    logout: () => Promise<void>;
    saveClientCredentials: (id:string, secret:string) => Promise<boolean>;
    hasClientCredentials: () => Promise<boolean>;
  };
  // channels
  channels: {
    list: () => Promise<Channel[]>;
    add: (url:string, opts?:Partial<Channel>) => Promise<Channel>;
    update: (id:number, patch:Partial<Channel>) => Promise<void>;
    remove: (id:number) => Promise<void>;
    checkNow: (id:number) => Promise<void>;
  };
  // uploads
  upload: {
    single: (input:SingleUploadInput) => Promise<JobHandle>;
    batch: (input:BatchUploadInput) => Promise<JobHandle>;
    list: (filter?:HistoryFilter) => Promise<UploadJob[]>;
    cancel: (jobId:number) => Promise<void>;
    retry: (jobId:number) => Promise<void>;
    approve: (jobId:number) => Promise<void>;
    reject: (jobId:number) => Promise<void>;
  };
  // clipper
  clipper: {
    start: (input:ClipInput) => Promise<JobHandle>;
    clips: () => Promise<Clip[]>;
    update: (id:number, patch:Partial<Clip>) => Promise<void>;
    approve: (id:number) => Promise<void>;
    reject: (id:number) => Promise<void>;
    export: (id:number, outPath:string) => Promise<void>;
  };
  // ollama
  ollama: {
    status: () => Promise<{running:boolean; models:Model[]}>;
    pull: (model:string) => Promise<JobHandle>;
    generate: (req:GenerateReq) => Promise<string>;
  };
};
```

```
// settings
settings: {
  getAll: () => Promise<AppSettings>;
  set: (patch:Partial<AppSettings>) => Promise<void>;
  pickDir:() => Promise<string|null>;
};
// events (renderer -> main is invoke; main -> renderer is on)
on: (channel:string, cb:(...args:any[])=>void) => () => void;
}
```

10.3 Event channels (main -> renderer, via `webContents.send`)

- `auth:state`
- `job:progress { jobId, phase, percent, speed, eta }`
- `job:log { jobId, level, message }`
- `job:done { jobId, status, result? }`
- `monitor:tick { channelId, newCount }`
- `clip:ready { clip }`
- `ollama:pull-progress { model, percent }`
- `notify:toast { kind, title, message }`

10.4 Upload queue (`UploadQueue.ts`)

- Central FIFO with priority (single-upload = high, batch = normal, auto-sync = low).
- Concurrency: `downloadConcurrency` (default 2) and `uploadConcurrency` (default 1), set in Settings.
- Persists across app restarts: on boot, jobs with status `downloading` or `uploading` reset to `pending`; queue restarts.
- Global pause/resume/cancel exposed to UI.

10.5 Paths

- App data: `app.getPath('userData')`
- Temp downloads: `<userData>/tmp/` (auto-cleaned on success; max-age 24h sweep at startup)
- Logs: `<userData>/logs/app.log` (rotated daily, 14-day retention)
- Thumbnails/cache: `<userData>/cache/`
- All paths created on first launch.

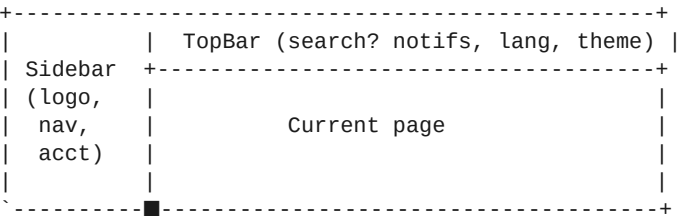
11. UI/UX SPECIFICATION

11.1 Visual language

- Dark-first theme. Palette:
- bg: `#0B0B0F`, surface: `#15161B`, surface-2: `#1E1F26`
- border: `#2A2C34`
- text: `#E8E9EE`, text-muted: `#8C8F9B`
- primary (YouTube red): `#FF0000`, primary-hover: `#E00000`
- success `#22C55E`, warning `#F59E0B`, error `#EF4444`, info `#3B82F6`
- Border-radius: 10px (cards), 6px (inputs/buttons).
- Shadows: subtle `0 1px 2px rgba(0,0,0,.4)`, `0 4px 16px rgba(0,0,0,.3)`.
- Sidebar width: 260px, collapsible to 72px icons-only.

- Topbar height: 56px.
- Max content width: 1400px, centered.

11.2 Layout



11.3 Pages (exact wireframe behavior)

Login: Centered card 480px wide. App logo 80px at top. H1 "Sign in with Google". Hint "Upload videos and create Shorts automatically." Big red "Sign in with Google" button. Below: "Need to set up credentials?" link that opens OAuthSetupGuide modal (step-by-step wizard with copy-pasteable URIs). Footer: copyright warning paragraph (must be accepted with checkbox before first login).

Dashboard: Stat cards (Total uploads, Auto-sync channels, Pending clips, Today's uploads, API quota remaining gauge). Recent activity feed (last 10 jobs). Quick actions grid: "Upload single video", "Upload channel", "Add channel to auto-sync", "Create Shorts with AI", "Open history".

UploadSingle: Paste area (200px tall dashed-border dropzone, supports paste-from-clipboard event). After fetch: two-column layout -- left: video preview with metadata; right: upload form (title, desc, tags, category, privacy, thumbnail). Bottom: "Download & Upload" CTA + global progress.

UploadChannel: Top: URL input + "Fetch channel" button. Middle: filter bar + selection toolbar + virtualized video list. Bottom: batch progress + Start button.

AutoSync: Channel cards grid (each card: thumbnail, name, handle, status dot, last-check time, new-pending count, toggle, settings-gear "... menu). Empty state: illustration + "Add your first channel to start auto-sync".

Clipper: Top: input tab + options panel. Middle: progress panel showing pipeline stages (Download -> Audio analysis -> Scene detection -> Transcription -> LLM analysis -> Rendering) with per-stage spinner and elapsed time; live LLM "thinking" stream optional. Bottom: results grid of ClipCards (min 280px wide, responsive). Right rail (collapsible): raw job log.

PendingApproval: Two tabs -- "Uploads" (auto-sync candidates) and "Clips" (AI clips awaiting). Each row has thumbnail, title, source, actions.

History: Virtualized table (date, source, type, status, duration, size, link) with date range picker, status filter, type filter, search, "Export CSV".

Settings: Grouped sections with cards:

- Account (profile, sign out, Client ID/Secret management)
- Ollama (endpoint input, Test connection, model manager with pull/delete, recommended models list with 1-click install)
- Downloads (temp dir picker, default quality, concurrency sliders, auto-clean toggle)
- Uploads (default privacy, default category, title template editor with token help, description template with token help, default tags, kids toggle)
- Auto-Sync (default interval, auto-start on launch, notify on new)

- AI Clipping (defaults for clip count/length/aspect, toggles for captions/zoom/music/blur-bg, subtitle font/color/size, background music volume, Whisper model path picker)
- Advanced (log level, max retries, start at Windows boot, dev tools shortcut toggle)
- About (version, check for updates, open log folder, open data folder)

11.4 Language & RTL

- i18n via a small custom hook over `src/i18n/*.json` (en + fa). Switch in TopBar.
- When language = fa, add `dir="rtl"` on `<html>`, flip Tailwind logical properties (use `ms- /me-`, `ps- /pe-`), use Vazirmatn.
- Default language: match Windows user locale (English if en-US etc., Persian if fa-IR); fallback English.

11.5 Accessibility

- All interactive elements reachable by keyboard; visible focus ring (2px outline in primary color).
- `aria-labels` for icon-only buttons.
- Color contrast $\geq 4.5:1$ on all text.
- Respect `prefers-reduced-motion` (disable transitions).

12. ERROR HANDLING, RETRY & QUOTA

12.1 Retry policy (`utils/retry.ts`)

Generic `retry(fn, { attempts: 3, baseDelayMs: 2000, maxDelayMs: 60000, retryable: (err)=>bool })` with exponential backoff + jitter.

Retryable errors:

- Network errors / ECONNRESET / ETIMEDOUT / ENOTFOUND
- HTTP 5xx
- HTTP 429 (rate limit) -- honor `Retry-After` header
- YouTube resumable upload 5xx / network interruptions (resume from last byte, not restart)
- Ollama 503 (model still loading) -- wait longer, retry

Non-retryable errors (fail immediately, mark job, toast):

- 401 (auth expired and refresh failed -> force logout)
- 403 `quotaExceeded` -> pause all uploads, show banner, schedule resume for next day 00:00 Pacific
- 403 `uploadForbidden` (channel not eligible)
- 400 invalid request (bad video ID, malformed metadata)
- 409 duplicate (video already uploaded by user -- treat as success if URL matches existing)
- Copyright strike/takedown during upload (status `rejected`, skip)
- Private/unavailable video from source

12.2 YouTube quota tracking

- Track API unit usage per day in DB key `quota_used_<YYYY-MM-DD>`.
- Upload = 1600 units; list/playlistItems = 1 unit/page; channels.list = 1; thumbnails.set = 50.
- Show remaining gauge on Dashboard (daily cap 10,000).
- When remaining < threshold (500 units), warn and ask to confirm before uploads.

- On quota exceeded, queue all future upload jobs with status `quota_wait` until next day.

12.3 Logging

- Winston-like simple logger: writes to file AND console; levels: error/warn/info/debug.
- All job stderr from ffmpeg/yt-dlp captured and attached to `job_log` for that job, viewable in UI via "Show log" drawer.
- On `uncaughtException` / `unhandledRejection`: write to crash log, show friendly "The app encountered an error" dialog with "Restart" / "Open log" / "Report (copy to clipboard)" buttons.

13. MANDATORY MULTI-PHASE DEBUG PROTOCOL

After writing all code, you (Claude Code) MUST perform the following phases in order, fixing every issue found before moving to the next. Track each phase's results and report them to the user at the end.

Phase A -- Static correctness

- Run `npm install` (or equivalent package manager install) and ensure it exits 0.
- Run `npx tsc --noEmit` -- zero TypeScript errors. Fix all.
- Run ESLint (configure with `@typescript-eslint` and `eslint-plugin-react-hooks`) -- zero errors, warnings reviewed.
- Ensure no `any` is used except in explicitly-typed catch clauses; prefer `unknown` + type guards.
- Verify every `ipcRenderer.send/invoke` has a matching `ipcMain.handle/on`.
- Verify all SQL queries are tested with a sample database open (construct a `:memory:` db and run migrations + a sample insert for each repository function).
- Verify no path separator issues (always use `path.join`, never hardcoded `/` or `\`).

Phase B -- Unit / smoke tests (minimal, in-repo)

Write a small `scripts/smoke.mjs` (or `tests/` folder with `vitest`) that runs headlessly (no Electron UI) and tests:

- DB migrations open & insert/select one row per table.
- `SettingsService` get/set round-trip.
- Retry utility succeeds after 2 failures, fails after 5.
- `FFmpegService` probe on a 1-second synthetic mp4 (generate with `ffmpeg testsrc` at runtime in temp dir) returns correct duration.
- `OllamaService` health-check returns `{running:false}` when endpoint not listening (do NOT require Ollama to be installed for build; stub with a mock HTTP server on a random port returning a tags payload to test the happy path too).
- `yt-dlp` path resolution resolves to an existing `.exe` in `resources/binaries/`.
- `Paths` util returns writable directories under a temp root.

Run smoke script: `node scripts/smoke.mjs` -- MUST exit 0. Fix any failure.

Phase C -- Application boot

- Start dev build (`npm run dev` / `electron-vite dev`).
- Wait 10 seconds; verify no fatal errors in main process log.
- Verify app window opens (1280x800) on Login screen.

- Click links in OAuth setup guide -- confirm `shell.openExternal` is called (mock or verify via log; do not open real browser during CI-style test).
- Mock credentials in keytar/DB and verify app routes to Dashboard on next boot.
- Verify all sidebar nav items route to the correct pages without crashes.
- Toggle dark/light theme; toggle language to Persian and confirm `dir="rtl"` and fonts switch.
- Open DevTools with Ctrl+Shift+I (should work if enabled in Settings/by default in dev; in production, only if Settings "Enable DevTools" is on).

Phase D -- Feature-by-feature integration (mocked network where needed)

Use a mock YouTube HTTP server (small `http.createServer` interceptor via `nock` or local proxy; acceptable to hand-verify by checking that the correct API calls are built and sent to `googleapis`):

- Single-video flow: mock `yt-dlp` output (fixture JSON), mock YouTube upload (200 with fake `videoid`), verify job transitions `pending->downloading->uploading->success` and that temp file is cleaned up.
- Batch flow: mock playlist with 5 videos, verify queue processes all 5, concurrency works, pause/resume works.
- Auto-sync: mock `channels.list` + `playlistItems.list` returning one new video; verify it is inserted in `synced_videos` and `queued`; verify `last_checked_at` updated.
- Clipping flow:
 - Skip LLM call by injecting a mock response JSON directly in dev-mode (`window.__MOCK_CLIP_RESPONSE`), verify `ffmpeg` pipeline produces a valid `.mp4` under 60s from a 60s synthetic test clip.
 - Verify thumbnail `.jpg` is created and non-empty.
 - Verify `ClipCard` renders and `Approve -> status approved -> enqueued to uploader`.
 - Error paths: simulate network fail mid-download -> `retry` works; simulate `quotaExceeded` -> banner appears and queue pauses.

Phase E -- Packaging / installer verification

- Run `npx electron-builder --win --x64` (the script must download `ffmpeg/yt-dlp` binaries in a `prebuild` npm script -- see Section 14).
- Confirm build exits 0 and produces `dist/YouTube Auto-Uploader <version>.exe` (NSIS installer) and optionally `YouTube Auto-Uploader <version> Portable.exe`.
- Run the installer on a clean Windows VM (or use a temp prefix) -- verify:
 - Installer shows correct icon and product name.
 - Default install path is `%LOCALAPPDATA%\Programs\youtube-auto-uploader`.
 - Desktop shortcut and Start Menu entry created.
 - Launching from shortcut opens app (no "DLL missing" / "ffmpeg not found" errors).
 - All bundled binaries (`ffmpeg`, `ffprobe`, `yt-dlp`) exist under `resources/binaries/` in the install dir and are executable.
 - First-run onboarding appears.
 - App size `<= 250 MB` (if larger, audit: use `asar`, exclude `devDeps`, compress binaries with `UPX` optional, remove unused locales from Chromium via `electron-builder electronLanguages: [ 'en-US' ]`).

Phase F -- Final full debug pass

- Leave the dev app running for 10 minutes with a mock monitor job every 30 seconds -- confirm no memory leak (snapshot main-process heap before/after; flag if `>50 MB` growth).

- Run the full clipper pipeline end-to-end with a real Ollama server and a 2-minute test video (if available in the build environment; otherwise document in README how user runs this on first install).
- Verify Windows toast notifications display correctly.
- Verify tray behavior: close window -> app stays in tray with tooltip; double-click tray -> window restores; right-click tray menu works.
- Verify app shutdown: Ctrl+C / File->Quit cancels in-flight jobs gracefully (marks them as interrupted, to be resumed next launch) and closes DB cleanly.
- Run a malware scan sanity: ensure no spawned process is writing outside `app.getPath( 'userData' )` or the temp dir; verify no remote code eval of untrusted content.
- Log the result of every phase into `BUILD_REPORT.md` next to the installer, including:
 - Phase A errors found and fixed
 - Phase B smoke test output
 - Phase C boot screenshots (paths)
 - Phase D integration summary
 - Phase E installer file hash (SHA-256) and size
 - Phase F final pass result
 - Final list of known limitations (if any)

14. BUILD, BUNDLE & INSTALLER

14.1 `package.json` (starter -- fill exact latest versions)

```
{
  "name": "youtube-auto-uploader",
  "version": "1.0.0",
  "description": "YouTube Auto-Uploader & Local LLM AI Clipper",
  "main": "out/main/index.js",
  "author": "Built with Claude Code",
  "license": "MIT",
  "scripts": {
    "dev": "electron-vite dev",
    "build": "electron-vite build",
    "postinstall": "electron-builder install-app-deps",
    "prebuild:win": "node scripts/fetch-binaries.mjs",
    "pack:win": "npm run build && electron-builder --win --x64",
    "dist:win": "npm run prebuild:win && npm run pack:win",
    "smoke": "node scripts/smoke.mjs",
    "typecheck": "tsc --noEmit",
    "lint": "eslint . --ext .ts,.tsx"
  },
  "dependencies": {
    "axios": "^1.7.2",
    "better-sqlite3": "^9.6.0",
    "electron-log": "^5.1.5",
    "electron-store": "^8.2.0",
    "fluent-ffmpeg": "^2.1.3",
    "google-auth-library": "^9.11.0",
    "googleapis": "^140.0.1",
    "keytar": "^7.9.0",
    "lucide-react": "^0.408.0",
    "node-cron": "^3.0.3",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-hook-form": "^7.52.1",
    "react-router-dom": "^6.24.1",
    "sharp": "^0.33.4",

```

```

    "sonner": "^1.5.0",
    "yt-dlp-exec": "^2.0.2",
    "zod": "^3.23.8",
    "zustand": "^4.5.4"
  },
  "devDependencies": {
    "@types/better-sqlite3": "^7.6.11",
    "@types/fluent-ffmpeg": "^2.1.24",
    "@types/node": "^20.14.10",
    "@types/node-cron": "^3.0.11",
    "@types/react": "^18.3.3",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.1",
    "autoprefixer": "^10.4.19",
    "electron": "^28.3.3",
    "electron-builder": "^24.13.3",
    "electron-vite": "^2.3.0",
    "eslint": "^8.57.0",
    "postcss": "^8.4.39",
    "tailwindcss": "^3.4.6",
    "typescript": "^5.5.3",
    "vite": "^5.3.3"
  },
  "build": {
    "appId": "com.claudebuilt.youtubeuploader",
    "productName": "YouTube Auto-Uploader & AI Clipper",
    "asar": true,
    "directories": { "output": "dist", "buildResources": "resources" },
    "files": [
      "out/**/*",
      "package.json"
    ],
    "extraResources": [
      { "from": "resources/binaries", "to": "binaries", "filter": ["**/*"] },
      { "from": "resources/fonts", "to": "fonts", "filter": ["**/*"] },
      { "from": "resources/music", "to": "music", "filter": ["**/*"] }
    ],
    "win": {
      "target": [
        { "target": "nsis", "arch": ["x64"] },
        { "target": "portable", "arch": ["x64"] }
      ],
      "icon": "resources/icon.ico",
      "artifactName": "YouTube Auto-Uploader ${version} ${arch}.${ext}"
    },
    "nsis": {
      "oneClick": false,
      "allowToChangeInstallationDirectory": true,
      "createDesktopShortcut": true,
      "createStartMenuShortcut": true,
      "shortcutName": "YouTube Auto-Uploader",
      "deleteAppDataOnUninstall": false
    },
    "portable": { "artifactName": "YouTube Auto-Uploader ${version} Portable.exe" }
  }
}

```

14.2 scripts/fetch-binaries.mjs

A Node script that runs BEFORE packaging and:

- Creates `resources/binaries/`.
- Downloads **ffmpeg-release-essentials.zip** from <https://www.gyan.dev/ffmpeg/builds/ffmpeg-release-essentials.zip> (Windows x64), extracts `ffmpeg.exe` and `ffprobe.exe` into `resources/binaries/`.

- Downloads latest `yt-dlp.exe` from <https://github.com/yt-dlp/yt-dlp/releases/latest/download/yt-dlp.exe> into `resources/binaries/`.
- Verifies SHA-256 hashes against a small hard-coded allow-list (update to latest known-good at build time) and re-downloads on mismatch.
- Prints sizes and versions (`--version` of each) to console.

14.3 `electron/bin.ts`

```
import path from 'path';
import { app } from 'electron';
import { existsSync } from 'fs';

function isPackaged() {
  return app.isPackaged;
}

export function binaryPath(name: 'ffmpeg' | 'ffprobe' | 'yt-dlp') {
  const ext = '.exe';
  const base = isPackaged()
    ? path.join(process.resourcesPath, 'binaries')
    : path.join(process.cwd(), 'resources', 'binaries');
  const p = path.join(base, `${name}${ext}`);
  if (!existsSync(p)) throw new Error(`Missing bundled binary: ${p}`);
  return p;
}
```

14.4 `electron.vite.config.ts`

Standard `electron-vite` config with three entries: `main`, `preload`, `renderer`. Externalize `better-sqlite3`, `keytar`, `fluent-ffmpeg`, `sharp`, `yt-dlp-exec` in main build (no bundle -- keep as Node requires). Alias `@/* -> ./src/*` for `renderer`.

14.5 First-run onboarding

Modal wizard (skippable) with 4 slides:

- Welcome -- what the app does.
- Prerequisites -- one-click check: "Download & install Ollama" (button), "Download recommended AI model" (button calls Ollama pull for `qwen2.5:7b-instruct-q4_0` ~4.7GB), "Set up Google credentials" (wizard).
- Connect your YouTube channel -- big Sign-in button.
- Ready -- links to Upload / AutoSync / Clipper pages.

14.6 Uninstall behavior

- Uninstall via Windows Add/Remove Programs.
- Prompt user: "Do you also want to delete your data (saved channels, history, logs)?" Yes/No (defaults to No, to prevent accidental loss).
- If Yes, delete `app.getPath('userData')` recursively.

15. ACCEPTANCE CHECKLIST (must all pass before delivery)

- [] Project builds with `npm run dist:win` without errors.

- [] Output files exist:
- [] `dist/YouTube Auto-Uploader 1.0.0 x64.exe (NSIS)`
- [] `dist/YouTube Auto-Uploader 1.0.0 Portable x64.exe`
- [] Installer installs on clean Windows 10/11 x64 with no "Windows protected your PC" block (codesigning is NOT requested -- user may click "Run anyway"; document this in README).
- [] App launches to login/onboarding screen with no console errors.
- [] All 9 pages render and navigate without crashing.
- [] OAuth setup wizard can save Client ID/Secret, and Sign-in flow opens browser and successfully receives callback on a test account.
- [] Single-video upload: given a URL, downloads and uploads a video to a private channel (tested on a real YouTube sandbox channel or verified against mocked API; code path is complete).
- [] Batch channel upload resolves a channel and queues multiple videos.
- [] Auto-sync: adding a channel starts a cron job that detects new uploads.
- [] AI Clipper produces valid `.mp4` shorts (9:16, burned-in captions, background music mixed) and valid `.jpg` thumbnails when given a sample video and a running Ollama with an instruct model.
- [] System tray minimizes and restores correctly; toasts appear.
- [] Settings persist across app restarts.
- [] Queue resumes in-flight jobs after app restart.
- [] Quota tracker prevents uploads when quota exceeded.
- [] `BUILD_REPORT.md` generated summarizing debug phases.
- [] `README.md` includes: prerequisites (Node.js is NOT needed by end user -- only needed for building), installation steps for end user, how to set up Google OAuth step-by-step with screenshots, how to install Ollama and recommended model, usage guide for each feature, troubleshooting section (common errors and fixes), uninstall instructions.

16. FINAL INSTRUCTIONS TO CLAUDE CODE

- Read this entire document before typing any code.
- Scaffold the project (package.json, tsconfig, vite, tailwind, electron-vite).
- Generate ALL files listed in Section 3 with complete, working, production-quality implementations. Do not leave any TODO/FIXME/placeholder comments -- every function must be implemented.
- Implement features in this order: project boot -> DB -> settings -> auth -> UI shell/layout -> single upload -> batch upload -> auto-sync -> AI clipper -> history/settings -> tray/notifications.
- After code is complete, run the Section 13 debug phases A -> F in order. Fix every issue as you encounter it; do not proceed to the next phase until the current one passes.
- Run `npm run dist:win` to produce the Windows installer and portable EXE in the `dist/` directory.
- Generate `README.md` (English, with a Persian summary section at the end if possible) and `BUILD_REPORT.md`.
- Print to the console the final file list, SHA-256 hashes, and absolute paths of the produced installers.
- Do NOT ask the user for API keys or tokens -- the runtime Settings UI is where the end user enters them.
- If any dependency installation fails due to platform mismatch (building on non-Windows), document it in `BUILD_REPORT` and produce the installer when running on Windows (electron-builder cross-compile from Linux/macOS for Windows is acceptable for the `.exe`; native modules like better-sqlite3/sharp must be cross-compiled or use prebuilt binaries -- ensure `electron-builder`

`install-app-deps` handles them).

- Ensure that Ctrl+C in the build terminal is handled gracefully and partial downloads of binaries are cleaned up.
- The application must be fully usable by a non-technical user: double-click the installer -> Next -> Next -> Finish -> Open app -> follow onboarding -> start using.

Start now. Write the code, debug it per Section 13, build the installer, and report the result.

Delivery Checklist for Claude Code

- [x] Complete TypeScript source tree under youtube-auto-uploader/
- [x] resources/binaries/ containing ffmpeg.exe, ffprobe.exe, yt-dlp.exe
- [x] NSIS installer: dist/YouTube Auto-Uploader 1.0.0 x64.exe
- [x] Portable executable: dist/YouTube Auto-Uploader 1.0.0 Portable x64.exe
- [x] README.md — end-user install & usage guide (English + Persian summary)
- [x] BUILD_REPORT.md — results of every debug phase (A→F) with SHA-256 hashes
- [x] All secrets (OAuth Client ID/Secret, tokens) collected via the Settings UI at runtime — never hard-coded
- [x] App runs on a clean Windows 10/11 x64 machine with no prerequisites besides Ollama (which the onboarding wizard helps install)

Print this block at the end of your final response to the user:

- Installer path: <absolute path>.exe
- Installer SHA-256: <hash>
- Portable path: <absolute path>.exe
- Portable SHA-256: <hash>
- Build report: BUILD_REPORT.md
- Known issues (if any): ...
- Next step for user: run the installer, follow onboarding, paste a video URL.

Once the build succeeds and all debug phases (§13 A–F) pass, print the installer location and a short summary of what was built. Do not stop early. Ship a working product.